GPU Driven Hair Rendering with the Mesh Shading Pipeline

Balázs Kovács[1]

[1]TU Wien

Abstract

The new mesh shading pipeline with the task and mesh shading stages offers unprecedented flexibility in processing and generating geometry. In this report I examine the use and benefits of replacing the traditional graphics pipeline with a GPU driven, mesh shading-based rendering pipeline for hair rendering.

1 Introduction

Historically, hair rendering methods heavily rely on the traditional graphics pipeline's geometry and tessellation shader stages for geometry processing and generation. Tessellation shaders while fast, heavily restrict what control we have over triangle generations, and geometry shaders use a single threaded programming model which is inefficient for modern GPUs.
In real-time graphics with hair simulation we have to rebuild the hair geometry for every single frame which is already fairly costly in of itself. However, besides geometry generation, hair rendering techniques also simulate light-scattering, self-shadowing and transparency, making them overall very expensive. Realistic hair models are commonly made up of strands and vertices in the order of 10^4 and 10^6 respectively, which adds further cost to rendering them.
The mesh shading pipeline is a natural fit for hair rendering. Exploiting its two stages and compute-like nature, we have the opportunity to build a more performant, GPU driven hair rendering pipeline.

2 GPU Driven Hair Rendering

The most important considerations when designing our mesh shading pipeline based hair renderer are the ideal workgroup size of our GPU and what work should one workgroup be responsible for. For NVIDIA GPUs this means a workgroup size of $w = 32$. The work done by a single mesh shader workgroup is heavily dependent on our representation of the hair model's data.
In my case, I decided on dividing hair strands into *strandlets*: subsections of a single hair strand with each strandlet consisting of w vertices at most. Mesh shader workgroups can then process one strandlet each, with each active thread outputting two vertices and indices for one quad (two primitives). To process an entire strand, multiple workgroups might be required. The task shader workgroups are responsible for launching the mesh shader workgroups, with each thread dispatching the right amount of workgroups required to render a single hair strand.
With this approach it becomes easy to handle varying strand lengths and strandlet counts as the task shader accounts for these. Finally, it is possible to initiate the entire hair rendering process by issuing a single command (*vkCmdDrawMeshTasks* for Vulkan) with a group size x of g_x to the GPU:

$$g_x = \left\lfloor \frac{n}{w} \right\rfloor \tag{1}$$

where n is the number of hair strands, and w is the workgroup size. The pipeline can be easily adopted to GPUs with other preferred workgroup sizes as the workgroup size only affects the strandlet generation process.

2.1 Hair Model and Representation

I decided on using the hair models made available by Cem Yuksel in the *.hair* format as a starting point. [1] However, since the pipeline uses vertex data to render hair it is possible to generate this data from an analytical definition of an arbitrary hair model. This introduces an extra discretization step into the preprocessing steps. Analytical definitions could also allow for more control of a Level of Detail generation process in the task shader.

After obtaining the array of vertices and strand length data, we must first process the data and create *"strand descriptions"*. A strand description contains the ID of a strand, how many vertices and strandlets it consists of and an offset in the vertex buffer. The pipeline will use two buffers as input data, one being the hair vertex data and the other being the strand description data.

2.2 Task and Mesh Shaders

Task Shader: As established previously, task shader threads will dispatch the amount of workgroups required to render a single hair strand. Aside from calculating this number, the workgroup will perform a parallel prefix scan, creating a table of strand offsets for the mesh shader workgroups. Along with the PPS table, the task shader payload also contains a global base strand offset.

In the future, camera to hair segment distance-based on-the-fly Level of Detail generation via per-hair strand interpolation could be implemented in the task shader. This interpolation would be crucial to avoid visually sharp bends in a strand as these become very visible close to the camera. Furthermore, since this method operates on a set of vertices and does not use guide strands or similar approximations this could be the point of control for dynamic geometry complexity.

Mesh Shader: Each workgroup will process strandlets with a maximum vertex count of 32, outputting 31 quads in total. Mesh shader workgroups work cooperatively, this allows for vertex reuse within the current strandlet with each thread generating 2 vertices. This means an output of 64 vertices and 62 triangles per workgroup, falling right within Nvidia's recommendations.
To generate quads, the mesh shader threads must calculate which strand and strandlet they are working on using the information provided by the task shader payload.
The current strand ID, α can be calculated as:

$$\alpha = b + k \tag{2}$$

where b is the base offset from the task payload, and k is which strand is being processed relative to b.

The strandlet ID, β can then be calculated as:

$$\beta = g + \epsilon_k \tag{3}$$

where g is the workgroup ID ($gl\_WorkGroupID.x$) and ϵ_k is the PPS table indexed by k.

And finally it is possible to calculate an offset δ into the vertex buffer as:

$$\delta = \Delta_\alpha + (\beta \cdot w) + l - \beta \tag{4}$$

where Δ_α is the base vertex buffer offset of the strand α and l is the current thread ID ($gl\_LocalInvocationID.x$).

We can then generate two vertices, each containing a position and tangent in world space.

Debug Rendering: There are various visualizations that can be useful for debugging the described hair rendering pipeline (see figure 1). We can choose to uniquely color each hair strand or strandlet. Furthermore, if we omit the re-use of vertices at quad borders we can assign different colors to specific quads.

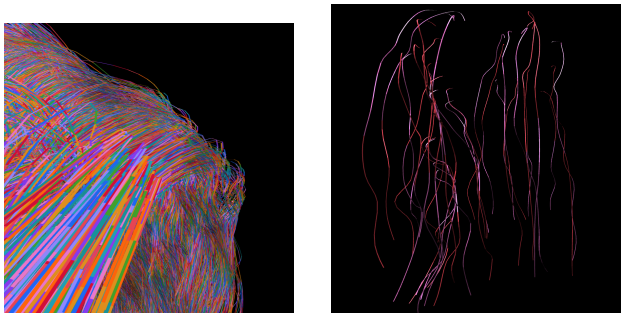

Figure 1: *Debug visualizations for the pipeline: quads (left) and strandlets (right).*

2.3 Shading Model

Currently, the pipeline uses the Kajiya-Kay shading model, which is a very simplistic shading model that treats hair strands as cylinders and uses the tangent for lighting computations. [2] This model does not account for the physical properties of hair strands. The average thickness of a hair strand is about 75 microns, which is small enough to the point where light not only bounces off the strand, but it can also pass through it. The amount of pigment (melanin) in a hair strand further influences this property. More melanin leads to darker hair color due to light absorption, which leads to less transparent hair strands.

In the future the current model should be replaced with one more faithful to the properties of hair. An idea for approximating this self-shadowing and transparent-like property of hair is to represent the density of the hair at a point in screen-space, this value could then be used to influence the lighting calculations. This could possibly implemented using a normal distribution function (NDF) computed for each point, which then can be convolved with an arbitrary bi-directional scattering distribution function (BSDF) to compute lighting.

3 Results

The method described above results in a hair rendering pipeline that is completely GPU driven and runs efficiently with good performance. Workgroup utilization is consistently above 90% for task and mesh shaders, leaving only a few threads idle. Therefore, work is distributed well across workgroups. The size of the task shader payloads and mesh shader outputs are well within Nvidia's recommendations and the pipeline can easily be modified to be optimal for

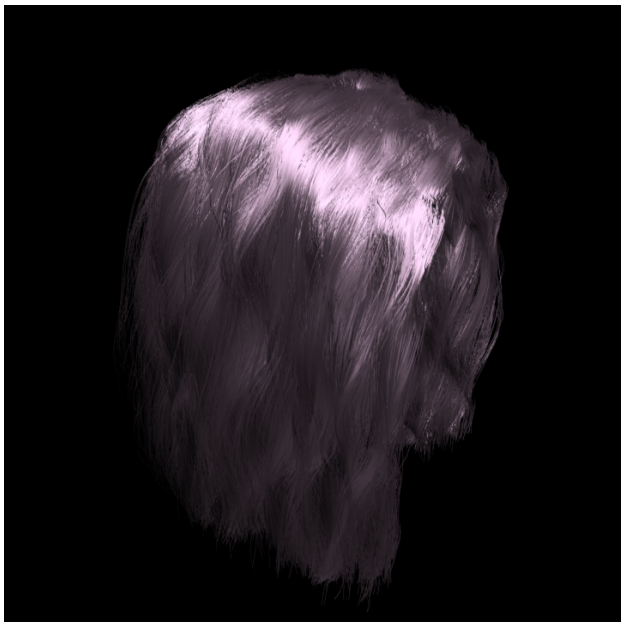

Figure 2: *Hair model rendered with the Kajiya-Kay model.*

AMD GPUs. I chose to implement the pipeline using Vulkan and GLSL, but there would be no obstacles to porting the code to shading language or an API with support for Mesh Shading such as DirectX/HLSL or Metal/MSL.

Measurements: In the following measurements I compare performance between the described pipeline and version without vertex reuse. The following measurements were done using an NVIDIA RTX 3060 Laptop GPU, and pipeline execution times were captured with NVIDIA NSight Graphics on a release build.

Table 1: *Benchmark results* without *vertex reuse*

Strand Count	Vertex Count	Time
50,000	1,250,000	2.60ms
50,000	2,450,000	6.20ms
50,000	3,441,580	9.04ms

It becomes clear that without implementing vertex reuse we run into bandwidth issues, with performance showing a seemingly exponential trend.

Table 2: *Benchmark results* with *vertex reuse*

Strand Count	Vertex Count	Time
50,000	1,250,000	2.38ms
50,000	2,450,000	3.64ms
50,000	3,441,580	5.13ms

From these measurements we can observe that performance scales linearly based on the number of vertices and can conclude that this method is suitable for use in real-time applications.

I also benchmarked the pipeline on an NVIDIA RTX 5070 GPU which is a newer desktop GPU.

Table 3: *Benchmark results (NVIDIA RTX 5070)*

Strand Count	Vertex Count	Time
50,000	1,250,000	0.52ms
50,000	2,450,000	0.69ms
50,000	3,441,580	0.72ms

Comparatively, the now old NVIDIA Hair SDK featured a tessellation shader-based hair rendering implementation for which in their white paper they claim 77 frames-per-second (FPS) for a hair model with around 18,000 strands with no mention of exact vertex count. [3] 77 FPS translates to ∼13ms, which is significantly slower even than the more complex models that were used for testing the presented pipeline.

A Note on Simulation: Incorporating hair physics simulation would be relatively straight forward as the pipeline relies on a buffer containing hair strand position data. Simulation could be implemented with 2 rotating vertex buffers: one is used by a compute pipeline that is executed asynchronously to calculate the positions for the next frame in one vertex buffer while the mesh shading pipeline utilizes the other vertex buffer to render the hair model.

References

[1] Cem Yuksel. *HAIR Model Files*. URL: https://www.cemyuksel.com/research/hairmodels/.

[2] J. T. Kajiya and T. L. Kay. "Rendering fur with three dimensional textures". In: *Proceedings of the 16th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '89. New York, NY, USA: Association for Computing Machinery, 1989, pp. 271–280. ISBN: 0897913124. DOI: 10.1145/74333.74361. URL: https://doi.org/10.1145/74333.74361.

[3] Sarah Tariq. "NVIDIA Hair SDK". In: (2010). URL: https://developer.download.nvidia.com/assets/gamedev/files/sdk/11/HairSDKWhitePaper.pdf.